

排序算法学习手册

本讲义面向正在学习排序算法的同学，配合课堂演示代码，帮助你从“故事 → 步骤 → 代码 → 复杂度”四个角度理解六大经典排序：插入、希尔、堆、选择、快速、归并。

👤 代码语言：C 📦 数据类型：int 数组 💡 建议搭配调试/打印一起食用

0. 总览 1. 插入排序 2. 希尔排序 3. 堆排序 4. 选择排序 5. 快速排序 6. 归并排序 7. 综合对比

0. 排序问题与六大算法地图

排序：给定一组可比较的元素（如 int 数组），将其按从小到大（或从大到小）重新排列。

在本讲义中，我们会依次学习 6 种**内部排序**（全部在内存中完成）：

插入排序 Insertion Sort

扑克牌摸牌的过程，适合
数据量小
或
基本有序
的情况。

希尔排序 Shell Sort

插入排序的“升级版”，通过
分组 + 缩小 gap
加速。

堆排序 Heap Sort

利用
堆结构
高效地不断选择最大值，时间复杂度稳定为 $O(n \log n)$ 。

选择排序 Selection Sort

每一趟找出最小/最大元素，逻辑非常直观，这里使用
双向选择
优化。

快速排序 Quick Sort

使用
分治 + 划分（挖坑法）
，平均性能非常优秀，是工程中常用的排序核心。

归并排序 Merge Sort

典型的
分治 + 归并
思路，稳定且时间复杂度始终为 $O(n \log n)$ 。

建议学习顺序：先理解思路简单的插入/选择，再到堆、快排、归并。希尔排序可以看作插入排序的“加速技巧”，可以放在插入后一起理解。

1. 插入排序 (Insertion Sort)

核心类比：整理手里的扑克牌 —— 始终保持左边一段是“排好序”的，把新牌插入到合适位置。

适合：数据量小 / 基本有序

稳定排序

原地排序 · $O(1)$ 额外空间

1.1 思想 & 步骤

- 分区：**逻辑上分成两部分：
 - 已排序区间：初始只有 $a[0]$
 - 未排序区间： $a[1] \dots a[n-1]$
- 依次摸牌：**外层 for 循环，每次把未排序区间的第一个元素 $a[i+1]$ 当作“新牌”。
- 向左找位置：**用指针 end 从已排序区间的右边开始，向左扫描，比新牌大的就**全部右移一格**。
- 插入落位：**循环结束时， $end + 1$ 就是新牌应该插入的位置。

1.2 为什么它是稳定的？

比较条件是 $a[end] > temp$ ，遇到**相等**时不会再移动，因此新元素会插入在“相等元素后面”，相等元素之间的相对顺序不会改变。

1.3 时间复杂度

- 最坏 / 平均： $O(n^2)$ —— 每次插入都要往前挪很多位置。
- 最好： $O(n)$ —— 原数组基本有序，内层 while 几乎不动。

练习建议：拿一组短数组（例如 $[2, 5, 3, 1]$ ），在草稿纸上手工模拟插入排序的过程，逐步写出每一趟的数组状态。

1.4 演示代码

```
insert_sort.c

void InsertRange(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int end = i;
        int temp = a[end + 1];

        while (end >= 0 && a[end] > temp)
        {
            a[end + 1] = a[end];
            end--;
        }

        a[end + 1] = temp;
        arrPrint(a, n);
    }
}
```

其中 arrPrint 为辅助函数，用于打印数组，可以帮助你观察每一趟排序后的结果。

平均时间	最好 / 最坏	空间复杂度	稳定性
$O(n^2)$	$O(n)$ / $O(n^2)$	$O(1)$	稳定 

2. 希尔排序 (Shell Sort)

插入排序的加速版：先用“大步长”让元素离目标位置更近，最后一步再用普通插入排序快速收尾。

- 适合：中等规模数据
- 不稳定排序
- 原地排序 · $O(1)$ 额外空间

2.1 思想 & 步骤

- 选择步长 gap**：简单版本从 $gap = n / 2$ 开始，每轮缩小一半，直到 $gap = 1$ 。
- 按照 gap 分组**：下标间隔为 gap 的元素看成一组，对这组做“插入排序”。代码中是通过 `for (int i = gap; i < n; i++)` 交织完成所有分组的插入过程。
- 不断缩小 gap**：每轮调整后，数组会“更接近有序”，最后当 $gap = 1$ 时，只需一次几乎是最佳情况的插入排序。

2.2 为什么它不稳定？

当 $gap > 1$ 时，元素可能被移动到很远的位置，**跨过**与它相等的元素，例如：

- 序列：[5_A, 7, 6, 2, 5_B, 4]
- 某一轮 $gap = 3$ 后，有可能变成 [2, 5_B, 4, 5_A, 7, 6]

可以看到 5_B 被调到了 5_A 前面，相对次序被打乱，因此不稳定。

2.3 时间复杂度

和选择的增量序列有关。课堂常用的简单取法（每次 $gap /= 2$ ）下，一般可认为平均复杂度约为 **$O(n^{1.3})$** 左右，比 $O(n^2)$ 有明显优势。

平均时间	最好 / 最坏	空间复杂度	稳定性
约 $O(n^{1.3})$ (与 gap 选取有关)	$O(n)$ / $O(n^2)$	$O(1)$	不稳定 

2.4 演示代码

shell_sort.c

```
void ShellSort(int a[], int n)
{
    for (int gap = n / 2; gap > 0; gap /= 2)
    {
        for (int i = gap; i < n; i++)
        {
            int temp = a[i];
            int end = i - gap;

            while (end >= 0 && a[end] > temp)
            {
                a[end + gap] = a[end];
                end -= gap;
            }

            a[end + gap] = temp;
        }
    }
}
```

注意这里的 `a[end + gap]` 和 `end -= gap`，与插入排序中的 `a[end + 1]` 和 `end--` 是“步长版”的对应关系。

3. 堆排序 (Heap Sort)

使用大顶堆不断“弹出”当前最大值，放到数组末尾，最终得到一个有序序列。

- 适合：需要 $O(n \log n)$ 且空间紧张
- 不稳定排序
- 原地排序 · 逻辑基于完全二叉树

3.1 思想 & 两个阶段

- 阶段一：建堆（大顶堆）**
 - 大顶堆：父节点值 $\geq$ 两个子节点值。
 - 最后一个非叶子结点下标是 $(size - 2) / 2$ 。
 - 从这个位置开始，往前遍历到 0，对每个结点做一次 AdjustDown，“向下调整成堆”。
- 阶段二：排序**
 - 此时 `a[0]` 是当前最大元素。
 - 把 `a[0]` 与末尾 `a[end]` 交换，最大元素就固定在末尾。
 - 缩小堆的有效大小 (`end--`)，再对 0 下标做一次 AdjustDown。
 - 重复上述过程，直到整个数组有序。

3.2 AdjustDown (向下调整) 逻辑

- 当前父节点下标：parent。
- 左子节点： $child = 2 * parent + 1$ 。
- 选出左右子节点中较大的一个。
- 如果 `a[child] > a[parent]`，交换，然后把 parent 移到 child 继续向下比较。

3.3 为什么不稳定？

交换堆顶和末尾元素时，相等元素可能会被跳到另一侧，从而改变原有的相对顺序，因此堆排序不稳定。

3.4 演示代码

heap_sort.c

```
void AdjustDown(int a[], int size, int root)
{
    int parent = root;
    int child = 2 * parent + 1;

    while (child < size)
    {
        if (child + 1 < size && a[child] < a[child + 1])
            child++;

        if (a[child] > a[parent])
        {
            Swap(a + child, a + parent);
            parent = child;
            child = parent * 2 + 1;
        }
        else break;
    }
}

void HeapSort(int a[], int size)
{
    for (int i = (size - 2) / 2; i >= 0; i--)
        AdjustDown(a, size, i);

    arrPrint(a, size);
}
```

```
int end = size - 1;
while (end > 0)
{
    Swap(&a[0], &a[end]);
    AdjustDown(a, end, 0);
    end--;
}

arrPrint(a, size);
}
```

若采用非递归版本的 AdjustDown，堆排序只使用常数级额外空间，因此空间复杂度为 O(1)。

时间复杂度

建堆 O(n) + n 次调整 O(log n) → O(n log n)

空间复杂度

O(1)

稳定性

不稳定 ❌

4. 选择排序 (Selection Sort)

每一趟从当前区间中找到最小值和最大值，把它们分别放到左边和右边。

适合：实现简单，便于教学演示

不稳定排序

原地排序 · O(1) 额外空间

4.1 思想 & 步骤 (双向选择版)

- 定义当前未排序区间：** [begin, end]，初始为 [0, size - 1]。
- 一趟扫描找最小和最大：**
 - 用 mini 记录最小值下标，maxi 记录最大值下标。
 - 遍历 i = begin ... end，不断更新 mini 和 maxi。
- 把最小值放到最左边：** 交换 a[begin] 和 a[mini]。
- 处理最大值下标的特殊情况：** 若 begin == maxi，说明原来的最大值被刚才的交换挪到了 mini 位置，此时需要修正 maxi = mini。
- 把最大值放到最右边：** 交换 a[end] 和 a[maxi]。
- 收缩区间：** begin++，end--，继续下一趟。

4.2 为什么不稳定？

最直观地看单向选择排序：如果数组为 [5_A, 5_B, 2]，第一趟找到最小值 2，会把 2 与 5_A 交换，得到 [2, 5_B, 5_A]，相等元素的相对顺序被打乱。

4.3 演示代码

```
select_sort.c

void SelectSort(int a[], int size)
{
    int begin = 0, end = size - 1;

    while (begin < end)
    {
        int mini = begin, maxi = begin;

        for (int i = begin; i <= end; i++)
        {
            if (a[i] < a[mini]) mini = i;
            if (a[i] > a[maxi]) maxi = i;
        }

        Swap(a + begin, a + mini);

        if (begin == maxi)
            maxi = mini;

        Swap(a + end, a + maxi);

        begin++;
        end--;
    }
}
```

与普通“单向选择”相比，这个版本一趟可以确定两个元素（最小和最大），效率稍高一些。

平均时间

O(n²)

最好 / 最坏

都是 O(n²)

空间复杂度

O(1)

稳定性

不稳定 ❌

5. 快速排序 (Quick Sort, 挖坑法)

分治法经典：选一个基准值，把数组划分为“比它小”和“比它大”两部分，再递归地分别排序。

适合：大规模数据

不稳定排序

平均 O(n log n) · 原地分区

5.1 挖坑法分区 PartSort1

- 选基准并挖第一个坑：**
 - 选择 a[begin] 为基准 key，记录下标 pivot = begin。
 - 可以理解为把 a[pivot] 挖成“坑”，准备让别的元素来填。

5.4 演示代码 (挖坑法)

```
quick_sort.c

int PartSort1(int a[], int left, int right)
{
    int begin = left, end = right;
    int pivot = begin;
```

2. **右边向左找小的填坑：** 从右侧 end 开始，向左移动，找到第一个 < key 的元素，把它放入当前的坑里，同时该位置变成新坑。
3. **左边向右找大的填坑：** 从左侧 begin 开始，向右移动，找到第一个 > key 的元素，把它放入当前的坑里，那个位置又成为新坑。
4. **左右交替，直到相遇：** 当 begin 与 end 相遇时，说明坑已经移动到了基准的最终位置。
5. **把 key 填回最后的坑：** 将基准 key 放入当前的坑里，返回该下标作为分割位置。

5.2 QuickSort 递归过程

- 递归出口：区间长度 ≤ 1 时，直接返回。
- 调用 PartSort1 得到基准位置 keyindex。
- 递归排序左右两半：[left, keyindex - 1] 和 [keyindex + 1, right]。

5.3 复杂度与优化

- 平均时间复杂度： $O(n \log n)$ ，在随机数据上表现很好。
- 最坏情况： $O(n^2)$ ，例如数组本身已经有序，而我们每次都选择最左端作为基准。
- 常见优化：随机化基准位置、三数取中、当区间很小时改用插入排序等。

```
int key = a[pivot];

while (begin < end)
{
    while (begin < end && a[end] >= key) end--;
    a[pivot] = a[end];
    pivot = end;

    while (begin < end && a[begin] <= key) begin++;
    a[pivot] = a[begin];
    pivot = begin;
}

a[pivot] = key;
return pivot;
}

void QuickSort(int a[], int left, int right)
{
    if (left >= right) return;

    int keyindex = PartSort1(a, left, right);

    QuickSort(a, left, keyindex - 1);
    QuickSort(a, keyindex + 1, right);
}
```

这里的 begin / end 始终包围着“坑”的位置，理解“坑”的移动有助于你跟踪整个分区过程。

平均时间	最坏时间	平均空间	稳定性
$O(n \log n)$	$O(n^2)$	$O(\log n)$ （递归栈）	不稳定 ❌

6. 归并排序（Merge Sort）

先把数组不断二分，直到每一段都“天然有序”（长度为 1），再一层一层地合并成更长的有序序列。

- 适合：大数据量，对稳定性有要求
- 稳定排序
- 需要额外 $O(n)$ 空间

6.1 思想 & 步骤

1. **分 (Divide) :**
 - 不断二分区间：[left, right] $\rightarrow$ [left, mid] 和 [mid+1, right]。
 - 当 left $\geq$ right 时，区间长度 ≤ 1 ，自然有序，递归返回。
2. **合 (Conquer & Merge) :**
 - 此时左右两段 [left, mid] 和 [mid+1, right] 已经各自有序。
 - 使用两个指针 begin1 / begin2 同步往右走，比较两侧当前元素，把较小者拷贝到 temp 中。
 - 一侧用完后，把另一侧剩余元素整体拷贝到 temp。
 - 最后把 temp[left...right] 复制回原数组的对应区间。

6.2 为什么是稳定的？

在合并时，当 $a[begin1] == a[begin2]$ ，代码中选择的是：

```
if (a[begin1] <= a[begin2]) ...
```

也就是**优先拷贝左边的元素**，因此左侧的“老元素”会排在右侧“新元素”之前，保持相等元素的相对顺序。

6.3 演示代码

```
merge_sort.c

void _MergeSort(int a[], int left, int right, int *temp)
{
    if (left >= right) return;

    int mid = (left + right) >> 1;

    _MergeSort(a, left, mid, temp);
    _MergeSort(a, mid + 1, right, temp);

    int begin1 = left, end1 = mid;
    int begin2 = mid + 1, end2 = right;
    int index = left;

    while (begin1 <= end1 && begin2 <= end2)
    {
        if (a[begin1] <= a[begin2])
            temp[index++] = a[begin1++];
        else
            temp[index++] = a[begin2++];
    }

    while (begin1 <= end1) temp[index++] = a[begin1++];
    while (begin2 <= end2) temp[index++] = a[begin2++];

    for (int i = left; i <= right; i++)
        a[i] = temp[i];
}

void MergeSort(int a[], int n)
{
    int* temp = (int*)malloc(sizeof(int) * n);
    _MergeSort(a, 0, n - 1, temp);
}
```

```
        free(temp);  
    }
```

为了节省空间，temp 只申请一次，并在递归过程中反复复用。整体需要 $O(n)$ 的额外空间。

时间复杂度	空间复杂度	稳定性
始终为 $O(n \log n)$	$O(n)$	稳定

7. 六大排序算法综合对比

这一节帮助你从“复杂度、空间、稳定性、是否原地”几个维度整体把握它们的优缺点。

排序算法	平均时间	最坏时间	最好时间	空间复杂度	稳定性	是否原地
插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定	是
希尔排序	约 $O(n^{1.3})$	$O(n^2)$	$O(n)$	$O(1)$	不稳定	是
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定	是
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定	是
快速排序	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(\log n)$ (递归栈)	不稳定	是
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定	否
稳定排序 ：相等元素排序前后的相对次序不变。						
原地排序 ：只使用常数级额外空间（不算递归栈）。						

- 建议你在 IDE 中逐行调试这些代码，观察数组在每一趟排序后的变化。
- 真正掌握一个排序算法的标志：你能不用看代码、用自己的话讲清楚它的思想和每一步在干什么。